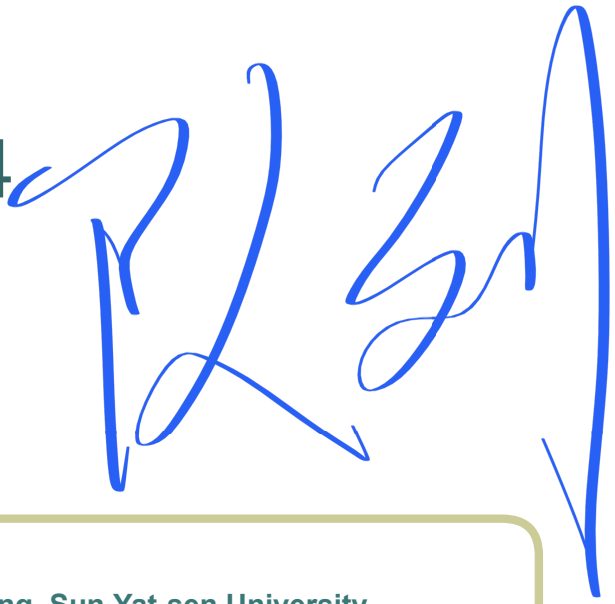
中山大學
SUN YAT-SEN UNIVERSITY

Lecture 4
Queues



Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- Specifications for Queues
- Implementations of Queues
- Circular Queues
- Application of Queues
  - Palindrome（回文）
  - Dual Stack
  - Breadth First Search

Specifications for Queues

- **Definition**: A queue is a list in which all additions to the list are made at one end, and all deletions from the list are made at the other end. Queues are also called ***first-in, first-out lists,*** or ***FIFO*** for short.



Specifications for Queues

- The first entry that will be removed from the queue, is called the ***front*** of the queue (or, sometimes, the ***head*** of the queue).
- The last entry in the queue, that is, the one most recently added, is called the ***rear*** (or the ***tail***) of the queue.

应用举例

- 队列在日常生活中和计算机程序设计中，有着非常重要的作用，在此，仅举出两个方面例子来说明它，其它应用在后面章节中将会遇到。
- 第一个例子就是CPU资源的竞争问题。在具有多个终端的计算机系统中，有多个用户需要使用CPU各自运行自己的程序，它们分别通过各自终端向操作系统提出使用CPU的请求，操作系统按照每个请求在时间上的先后顺序，将其排成一个队列，每次把CPU分配给队头用户使用，当相应的程序运行结束，则令其出队，再把CPU分配给新的队头用户，直到所有用户任务处理完毕。
- 第二个例子就是主机与外部设备之间速度不匹配的问题。以主机和打印机为例来说明，主机输出数据给打印机打印，主机输出数据的速度比打印机打印的速度要快得多，若直接把输出的数据送给打印机打印，由于速度不匹配，显然是不行的。所以解决的方法是设置一个打印数据缓冲区，主机把要打印输出的数据依此写如到这个缓冲区中，写满后就暂停输出，继而去做其它的事情，打印机就从缓冲区中按照先进先出的原则依次取出数据并打印，打印完后再向主机发出请求，

Queues in STL

- The std::queue class template is a *container adaptor* that gives the functionality of a queue - specifically, a FIFO (first-in, first-out) data structure. The queue pushes the elements on the back of the underlying container and pops them from the front.

template <class T, class Container = std::deque<T> > class queue;

```cpp
#include <cassert>
#include <iostream>
#include <queue>

int main()
{
    std::queue<int> q;

    q.push(0); // back pushes 0
    q.push(1); // q = 0 1
    q.push(2); // q = 0 1 2
    q.push(3); // q = 0 1 2 3

    assert(q.front() == 0);
    assert(q.back() == 3);
    assert(q.size() == 4);

    q.pop(); // removes the front element, 0
    assert(q.size() == 3);

    // Print and remove all elements. Note that std::queue does not
    // support begin()/end(), so a range-for-loop cannot be used.
    std::cout << "q: ";
    for (; !q.empty(); q.pop())
        std::cout << q.front() << ' ';
    std::cout << '\n';
    assert(q.size() == 0);
}
```

Output:

```
q: 1 2 3
```

Queue operations

```
Queue :: Queue( );
```
Post: The Queue has been created and is initialized to be empty.

Constructor
构造函数

```
Error_code Queue :: append(const Queue_entry &x);
```
Post: If there is space, x is added to the Queue as its rear. Otherwise an Error_code of overflow is returned.

Insertion / Enqueue
入队

```
Error_code Queue :: serve( );
```
Post: If the Queue is not empty, the front of the Queue has been removed. Otherwise an Error_code of underflow is returned.

Deletion / Dequeue
出队

```
Error_code Queue :: retrieve(Queue_entry &x) const;
```
Post: If the Queue is not empty, the front of the Queue has been recorded as x. Otherwise an Error_code of underflow is returned.

Get the front
取队头元素

```
bool Queue :: empty( ) const;
```
Post: Return true if the Queue is empty, otherwise return false.

Check emptiness
检查队是否空

Extended Queues

```
class Extended_queue: public Queue {
public:
    bool full() const;
    int size() const;
    void clear();
    Error_code serve_and_retrieve(Queue
};
```

bool Extended_queue :: full( ) const;

Post: Return **true** if the Extended_queue is full; return **false** otherwise.

void Extended_queue :: clear( );

Post: All entries in the Extended_queue have been removed; it is now empty.

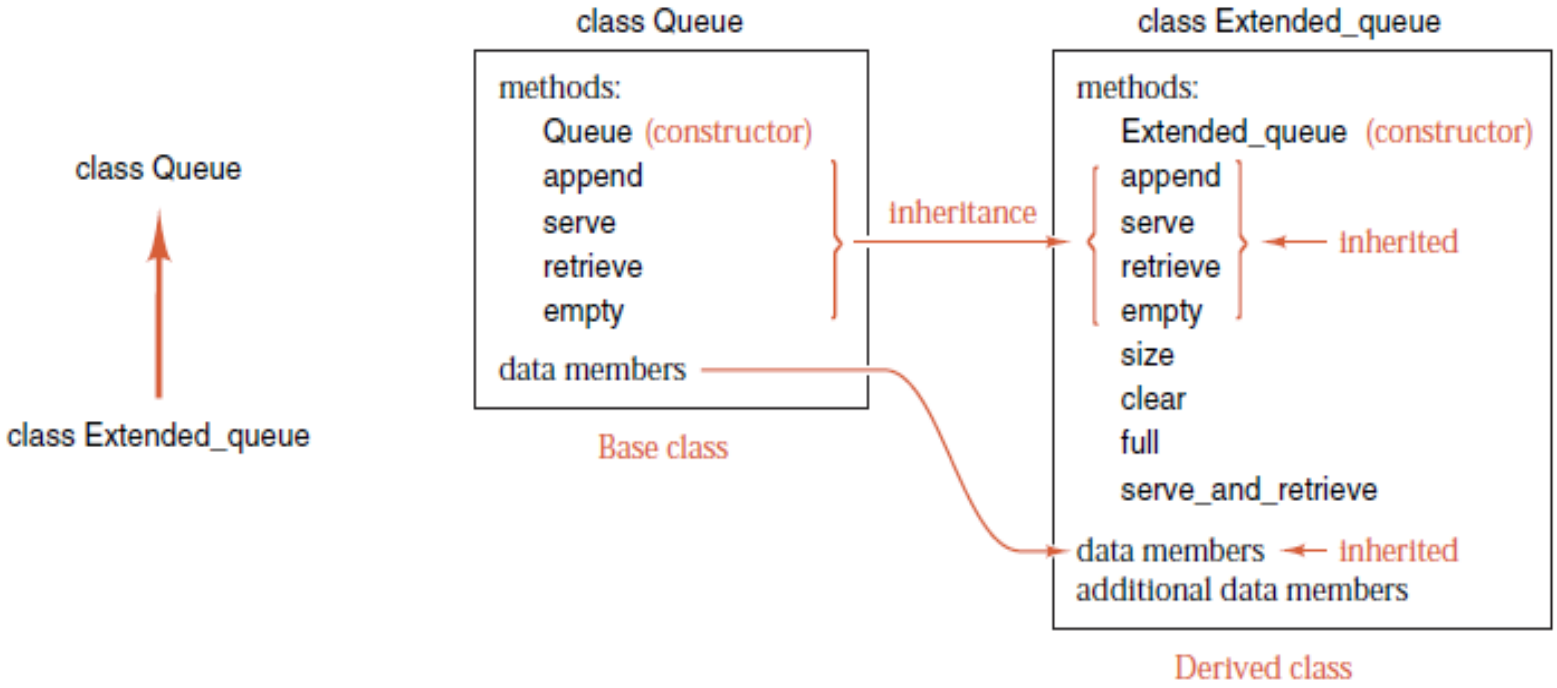
int Extended_queue :: size( ) const;

Post: Return the number of entries in the Extended_queue.

Error_code Extended_queue ::
serve_and_retrieve(Queue_entry &item);

Post: Return underflow if the Extended_queue is empty. Otherwise remove and copy the item at the front of the Extended_queue to item and return success.

Inheritance



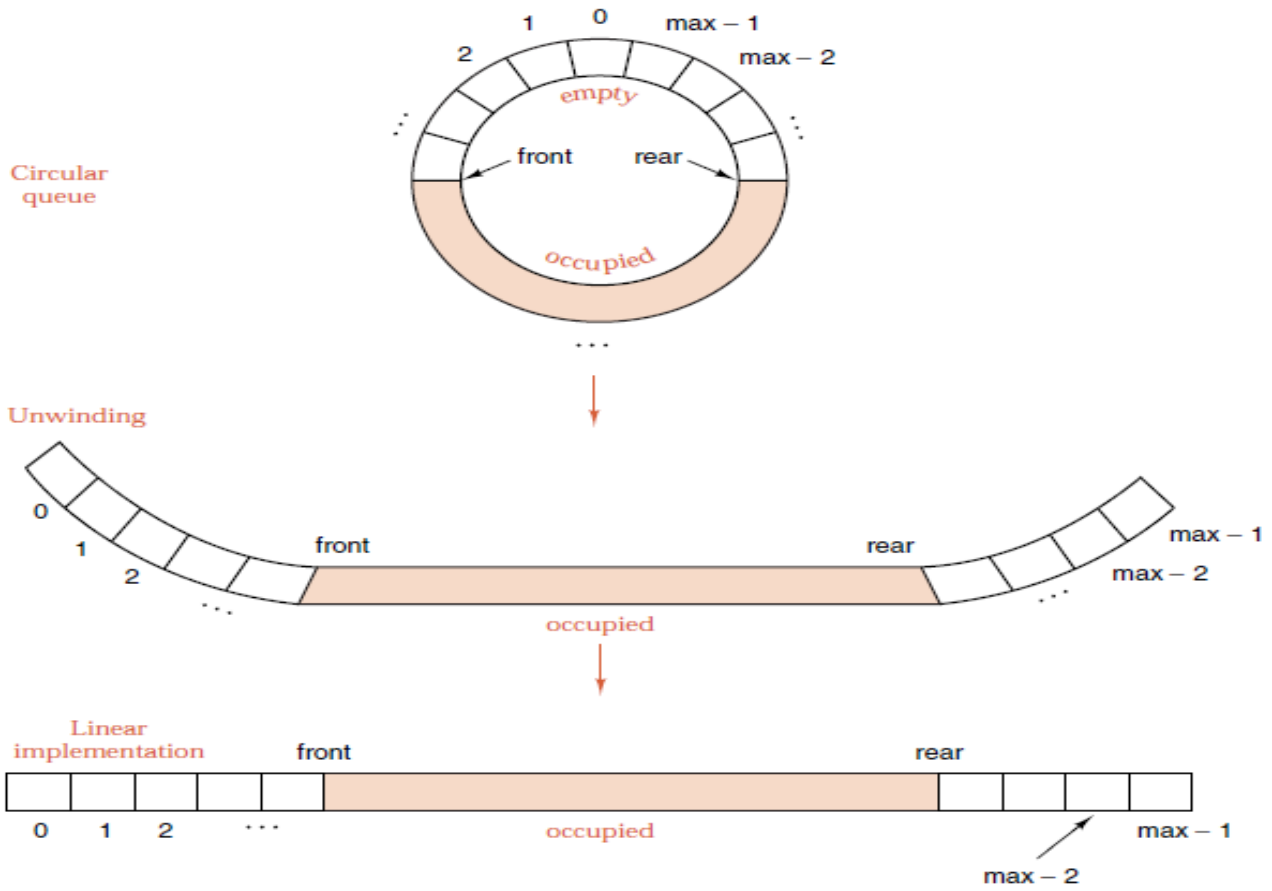
(a) Hierarchy diagram

(b) Derived class Extended_queue from base class Queue

Implementations of queues

- The Physical Model
  - A linear array with *the* front always in the first position and all entries moved up the array whenever the front is removed.
- Linear Implementation
  - Two indices to keep track of both the front and the rear of the queue
  - To serve an entry, take the entry and increase the front by one
  - To append an entry to the queue, increase the rear by one and put the entry in that position
  - **Problem**: cannot reuse the discarded space
  - When the queue is regularly emptied, this is good.

Circular implementation of queues

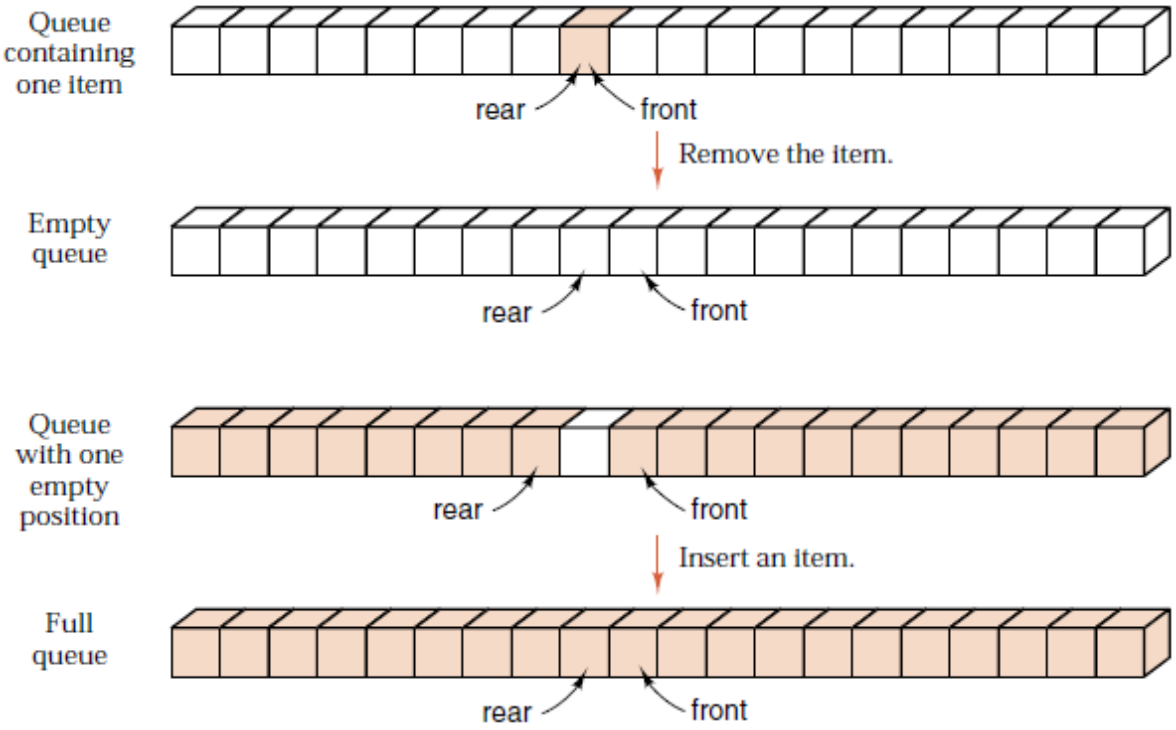


Circular arrays

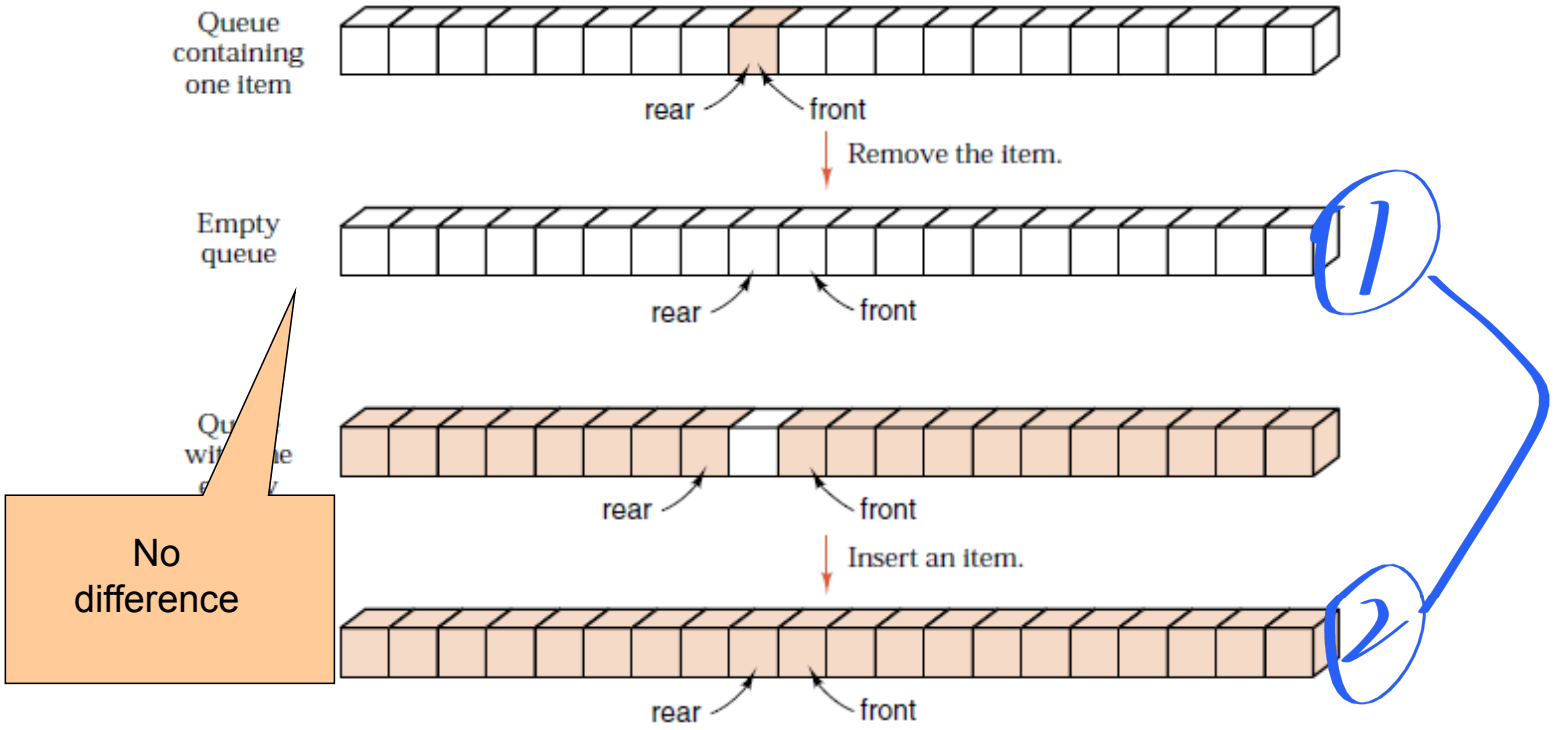
- 将数组设想为一个循环的，而非线性的；
- 用两个下标front和rear记录队头和队尾位置；
- 添加元素时，rear右移，将元素置于rear位置。当rear等于max时(last index), rear 置 0.
- 元素出队时，删除位于front位置的元素，然后front右移. 当front 等于 max时, 置 front 为 0。

- Equivalent methods to increment an index i in a circular array:
  - i = ((i + 1) == max) ? 0 : (i + 1);
  - if ((i + 1) == max) i = 0; else i = i + 1;
  - i = (i + 1)%max;

Boundary conditions

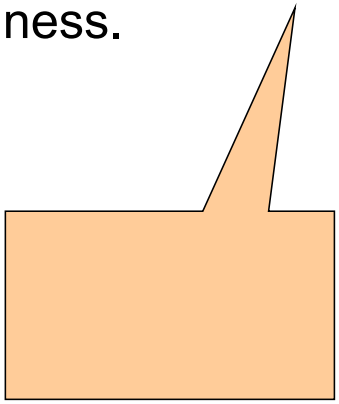


Boundary conditions



Deal with boundary conditions

- A circular array with front and rear indices and one position left vacant.
- A circular array with front and rear indices and a Boolean flag to indicate fullness (or emptiness).
- A circular array with front and rear indices and an integer counter of entries.
- A circular array with front and rear indices taking special values to indicate emptiness.



Class definition

```
const int maxqueue = 10; // small value for testing
class Queue {
public:
    Queue( );
    bool empty( ) const;
    Error_code serve( );
    Error_code append(const Queue_entry &item);
    Error_code retrieve(Queue_entry &item) const;
protected:
    int count;
    int front, rear;
    Queue_entry entry[maxqueue];
};
```

enum Error_code {success, underflow, overflow};

Programming Precept

Practice information hiding:
Separate the application of data structures from their implementation.

- Note:
  - Use an integer counter of entries
  - The keyword **protected**

Queue member functions

```
Queue :: Queue( )
/* Post: The Queue is initialized to be empty. */
{
    count = 0;
    rear = maxqueue - 1;
    front = 0;
}

bool Queue :: empty( ) const
/* Post: Return true if the Queue is empty, otherwise
    return false. */
{
    return count == 0;
}
```

通过count来判断满

Queue member functions

```
Error_code Queue :: append(const Queue_entry &item)
/* Post: item is added to the rear of the Queue. If the Queue is full return an
Error_code of overflow and leave the Queue unchanged. */
{
   if (count >= maxqueue) return overflow;
   count++;
   rear = ((rear + 1) == maxqueue) ? 0 : (rear + 1);
   entry[rear] = item;
   return success;
}
Error_code Queue :: serve( )
/* Post: The front of the Queue is removed. If the Queue is empty return an
Error code of underflow. */
{
   if (count <= 0) return underflow;
   count--;
   front = ((front + 1) == maxqueue) ? 0 : (front + 1);
   return success;
}
Error_code Queue :: retrieve(Queue_entry &item) const
/* Post: The front of the Queue retrieved to the output parameter item. If
the Queue is empty return an Error_code of underflow. */
{
   if (count <= 0) return underflow;
   item = entry[front];
   return success;
}
```

Exercise:

- Use a Boolean flag to implement Circular Queues.
- What is the relation of *front* and *rear* in the boundary condition?

Application of Queues: Palindrome

- A palindrome is a word, phrase, number, or other sequence of symbols or elements that reads the same forward or reversed.
- For example, “abba” and “abcba” are palindromes, but “abcde” or “ababab” are not palindromes.
- Please write an algorithm to check whether a string is a palindrome.

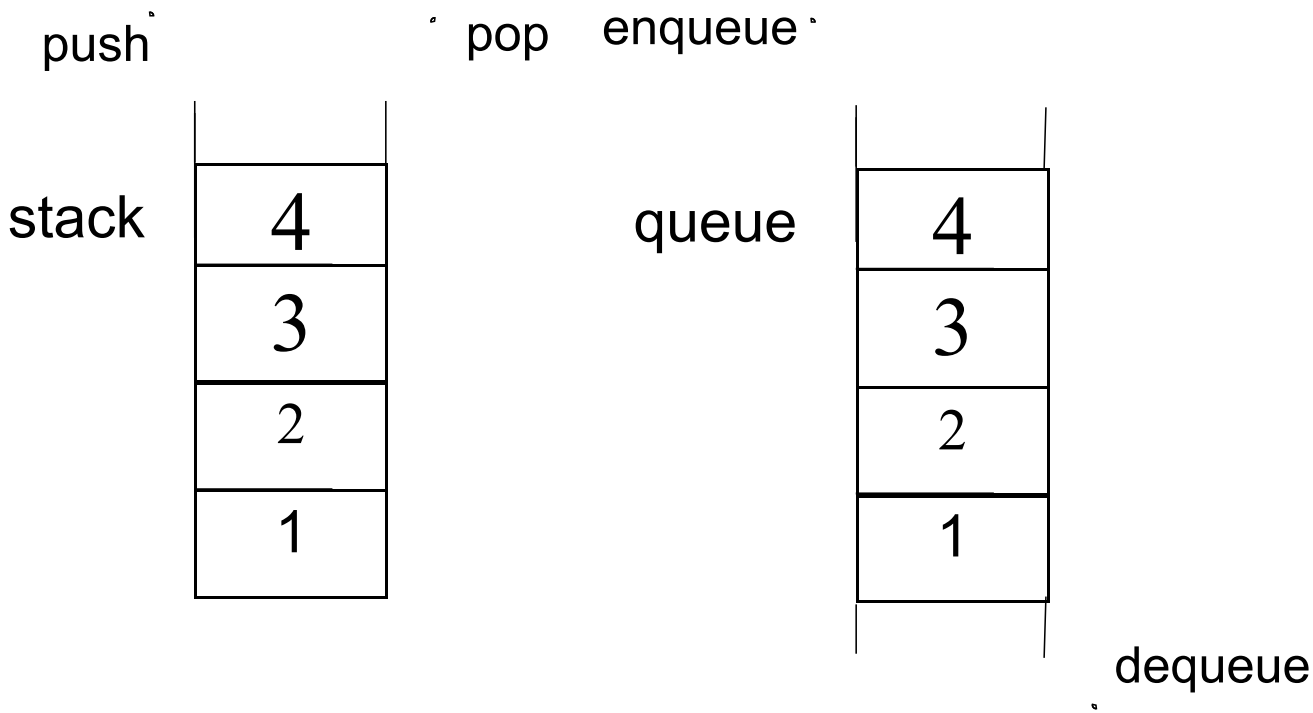
Application of Queues: Palindrome

- Basic idea: Use “stack” and “queue” simultaneously.

```
bool Palindrome_Test(char *str)
//判别输入的字符串是否回文序列,是则返回true,否则返回false
{
    InitStack(S);
    InitQueue(Q);
    for (char *c = str; *c != '\0'; c++) {
        Push(S, *c);
        EnQueue(Q, *c);   //同时使用栈和队列两种结构
    }
    while(!StackEmpty(S)) {
        Pop(S, a);
        DeQueue(Q, b);
         if (a != b) return false;
     }
   return true;
}
```

Application of Queues: Dual Stack

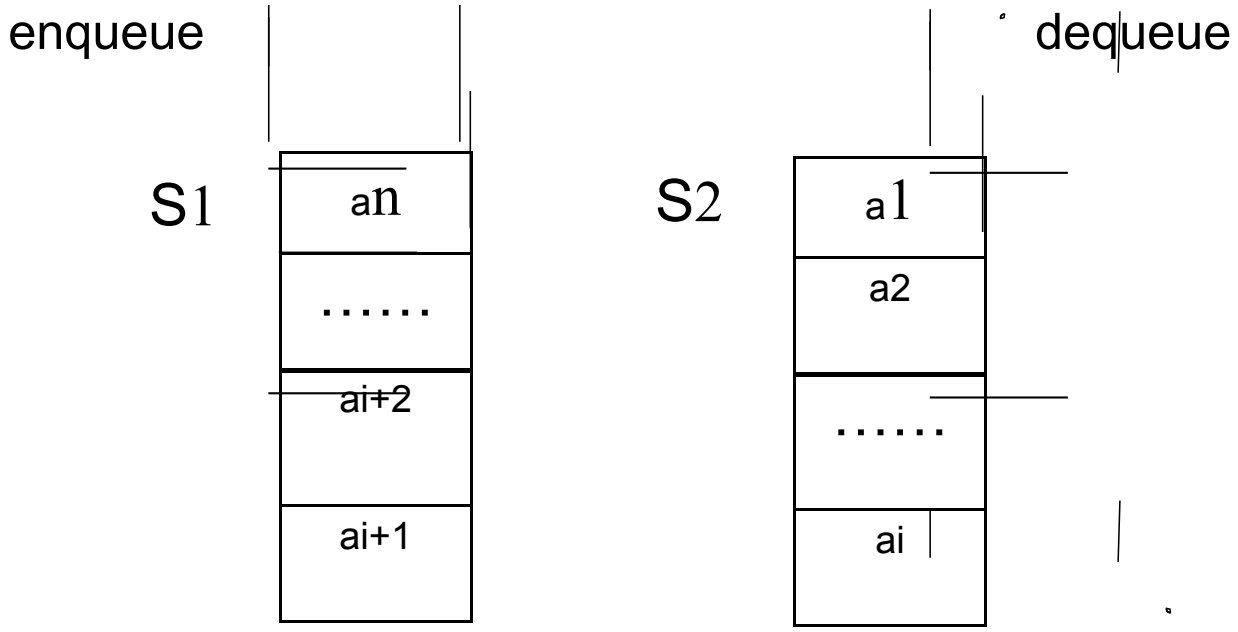
- 利用两个栈S1和S2模拟一个队列，写出入队和出队的算法(可用栈的基本操作)。(某年研究生入学试题)



Application of Queues: Dual Stack

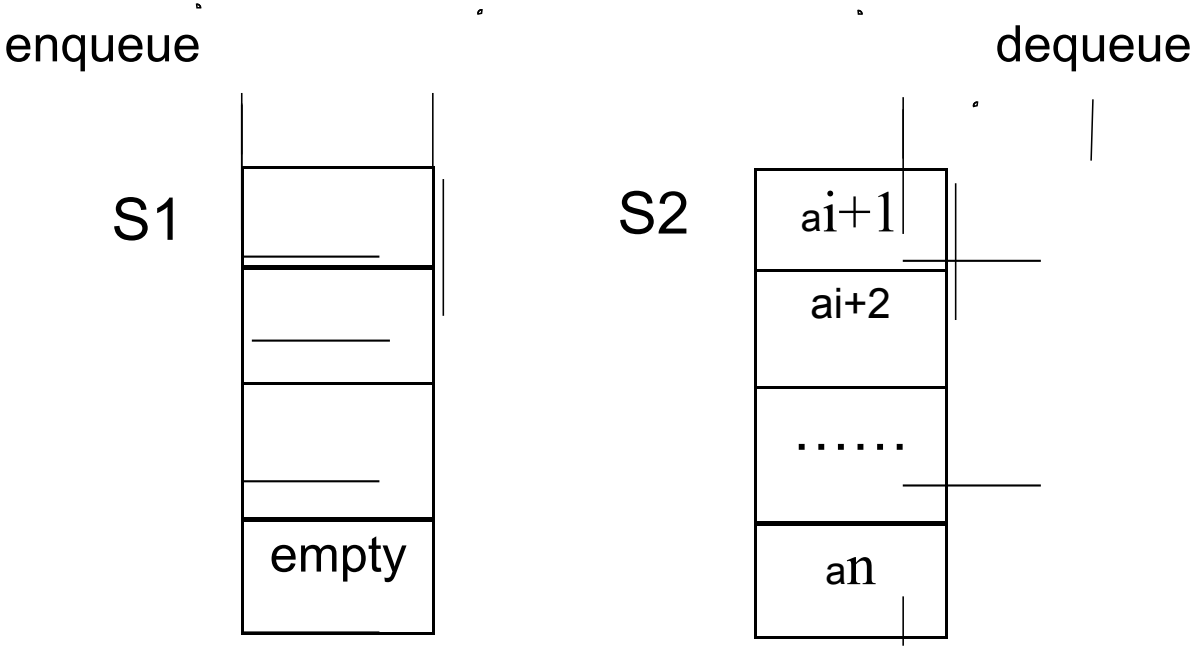
- Algorithm overview

S1 for enqueue, S 2 for dequeue



Application of Queues: Dual Stack

- For dequeue operation, pop S2 if S2 is not empty, otherwise, transfer elements from S1 to S2.



Application of Queues: Dual Stack

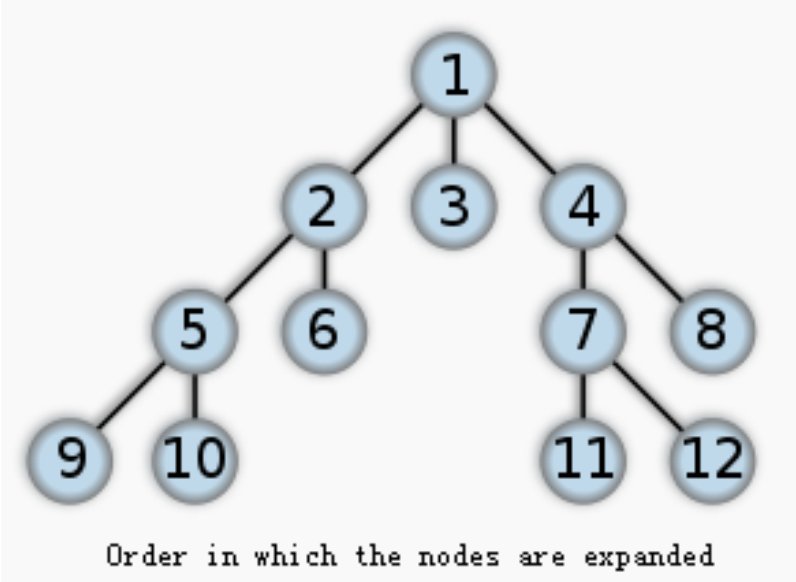
```
Error_code enqueue(DualStack Q, DataType a )
{
    if(!Q.StackFull(S1)) Q.push(S1,a);
    else if(Q.StackEmpty(S2))
        {
            while(!Q.StackEmpty(S1))
               Q.push(S2, Q.pop(S1));
            Q.push(S1,a);
         }
        else return overflow;
    return success;
}
```

Application of Queues: Dual Stack

```
Error_code dequeue(DualStack Q ,DataType a)
{
  if(!(Q.StackEmpty(S1)& &Q.StackEmpty(S2)))
  {   if(!Q.StackEmpty(S2))
          a=Q.pop(S2);
      else {
           while(!Q.StackEmpty(S1)) Q.push(S2,Q.pop(S1));
           a=Q.pop(S2);
      }
      return success;
  }
  else return underflow;
}
```

Application of Queues: Breadth First Search

- **Breadth-first search (BFS)** is a strategy for searching in a graph when search is limited to essentially two operations: (a) visit and inspect a node of a graph; (b) gain access to visit the nodes that neighbor the currently visited node. The BFS begins at a root node and inspects all the neighboring nodes. Then for each of those neighbor nodes in turn, it inspects their neighbor nodes which were unvisited, and so on.



Application of Queues: Breadth First Search

- The algorithm uses a queue data structure to store intermediate results as it traverses the graph, as follows:
- Enqueue the root node
- Dequeue a node and examine it
  - If the element sought is found in this node, quit the search and return a result.
  - Otherwise enqueue any successors (the direct child nodes) that have not yet been discovered.
- If the queue is empty, every node on the graph has been examined – quit the search and return "not found".
- If the queue is not empty, repeat from Step 2.

Application of Queues: Breadth First Search

```
1  procedure BFS(G, v) is
2      create a queue Q
3      create a set V
4      add v to V
5      enqueue v onto Q
6      while Q is not empty loop
7          t ← Q.dequeue()
8          if t is what we are looking for then
9              return t
10         end if
11         for all edges e in G.adjacentEdges(t) loop
12             u ← G.adjacentVertex(t, e)
13             if u is not in V then
14                 add u to V
15                 enqueue u onto Q
16             end if
17         end loop
18     end loop
19     return none
20 end BFS
```

Abstract Queues

DEFINITION A ***queue*** of elements of type T is a finite sequence of elements of T together with the following operations:

1. *Create* the queue, leaving it empty.
2. Test whether the queue is *Empty*.
3. *Append* a new entry onto the rear of the queue, provided the queue is not full.
4. *Serve* (and remove) the entry from the front of the queue, provided the queue is not empty.
5. *Retrieve* the front entry off the queue, provided the queue is not empty.

DEFINITION An ***extended queue*** of elements of type T is a queue of elements of T together with the following additional operations:

4. Determine whether the queue is *full* or not.
5. Find the *size* of the queue.
6. *Serve and retrieve* the front entry in the queue, provided the queue is not empty.
7. *Clear* the queue to make it empty.

Thank you!

